# 7 Academic Simulation Website

The academic website makes the dual-queue model inspectable through an English-language interface. Users can vary entry arrival rates, class-ending surges, walking assumptions, lift capacities and service cycles, and replay the resulting queues. A fixed seed reproduces a run. The public implementation and source are available online.[1]

**Computation and presentation.** The numerical engine in `backend/` is separated from controls, animation and plots in `frontend/`. A versioned Web Worker interface exposes three operations: `generate` returns replay frames; `inspect` returns cost components at a selected state; and `collect` builds conditional tables. The frontend displays returned values rather than independently implementing the choice equations. Workers keep computation off the main interface thread; a separate collection worker allows replay inspection during table generation. Static GitHub Pages hosting is sufficient, without an HTTP application server or database. This website backend is JavaScript, distinct from any separately developed team SimPy implementation.

**Observation and explanation.** Replay distinguishes physical queues from people walking to their chosen destination. A passenger joins a queue only on reaching its tail, and boarding follows individual lift events. Inspection separates walking time, expected position loss and batch-service waiting time. It explains why a shorter-looking queue can remain unattractive once switching costs are included. The corridor drawing is illustrative, rather than evidence of measured pedestrian geometry.

**Partial-information lookup.** The lookup conditions on entry side, time window and observed local queue length to estimate the opposite queue's empirical distribution. It compares conditional mean costs using the same cost function as the full-information engine, without inspecting the hidden live queue. Missing conditions remain missing, and fewer than 30 matching snapshots produces no definite recommendation. Changing parameters invalidates the table. CSV/JSON exports preserve parameters, seed, bins, counts and cost sums for reproducibility.

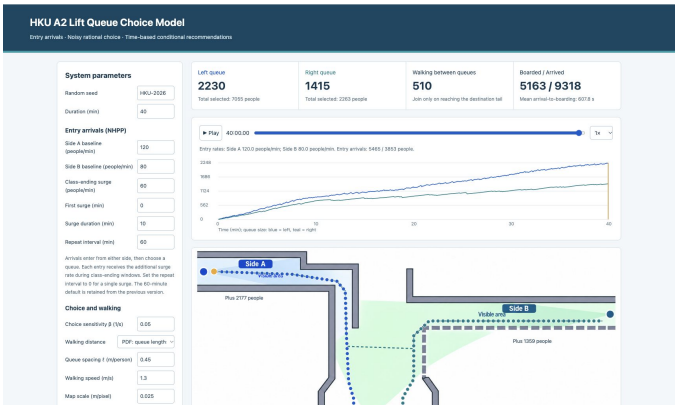



Figure 1: Local website replay with illustrative default inputs. Queue counts and displayed delays are simulated, not field measurements.

[1] Live website; source repository.

# 9 Experiments and Results

## 9.1 Design and reproducibility

These experiments evaluate the current website engine under stated assumptions, rather than calibrating it to field observations. We compare lower-demand arrival rates of 40 and 30 people/minute at entries A and B with the defaults of 120 and 80. Each baseline is run with and without a surge adding 60 people/minute to *each* entry during minutes 0–10. Each of the four scenarios has 30 independently seeded, empty-start, 40-minute trajectories. States are recorded every five seconds; events between observations are still processed at their scheduled times.

Fixed parameters are four lifts per bank, capacity limit 20, mean cycles 40 and 200 seconds, cycle standard deviations 4 and 20 seconds, speed 1.3 m/s, spacing 0.45 m/person and Logit sensitivity $\beta = 0.05\,\mathrm{s}^{-1}$. Trip capacity uses the implemented discrete distribution on 16–20, with mean 18. Cycle draws are truncated below ten seconds. The queue-length walking rule is used, rather than the optional mapped-route rule.

The primary outcome is the number not boarded by minute 40, including people queueing and walking. We also record arrivals, boarding fraction and mean arrival-to-boarding time among completed passengers. The latter excludes unfinished waits and is not the mean for all arrivals. Intervals use independent trajectory outcomes, $\bar{x} \pm t_{29,.975} s/\sqrt{30}$; within-run snapshots are not treated as independent replications.

## 9.2 Demand and finite-horizon backlog

The approximate nominal total service rate is

$$\mu_{\text{total}} = 60\left(\frac{4 \times 18}{40} + \frac{4 \times 18}{200}\right) = 129.6 \text{ people/minute}.$$

The default baseline of 200 therefore exceeds nominal service before any surge; 70 leaves recovery capacity after the surge ends. The ten-minute surge adds an expected 1,200 entrants across the two entries.

| Baseline | Surge | Arrivals | Not boarded at 40 min | Boarded delay (s) |
|---|---|---|---|---|
| 70/min | No | 2,805.7 | $16.4 \pm 3.6$ | 14.3 |
| 70/min | Yes | 3,999.8 | $15.1 \pm 3.7$ | 102.9 |
| 200/min | No | 7,985.2 | $2,816.5 \pm 33.7$ | 402.6 |
| 200/min | Yes | 9,205.3 | $4,028.5 \pm 41.7$ | 609.2 |

Table 1: Means over 30 trajectories. The ± entries are 95% t-interval half-widths for final backlog only. Delay includes only passengers boarded before the horizon.

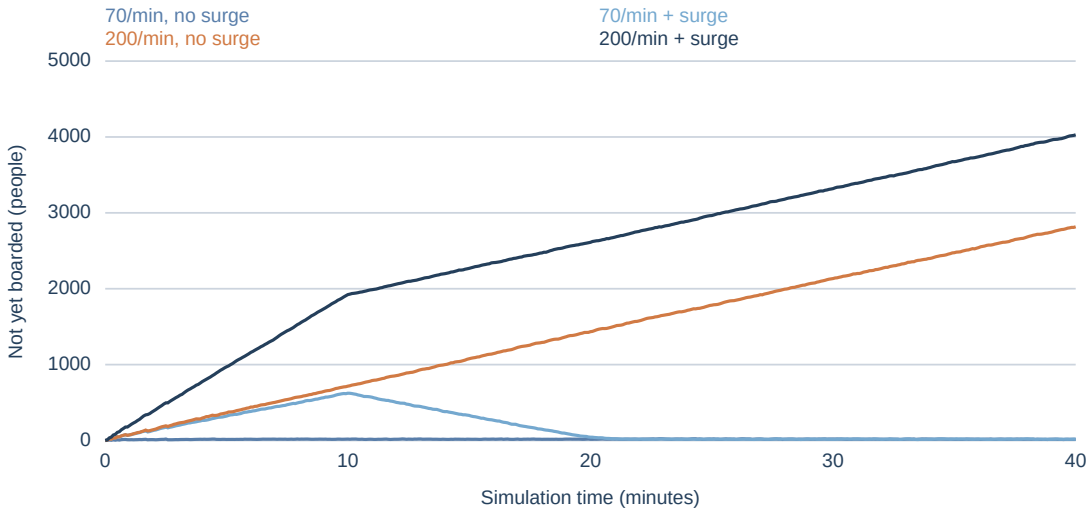



Figure 2: Mean number not yet boarded, including queued and walking passengers. All scenarios start empty.

## 9.3 Interpretation and conditional-table checks

At lower baseline demand, the surge raises completed-passenger mean delay from 14.3 to 102.9 seconds, but final backlog remains approximately 15–16 people. The trajectory plot shows recovery within the horizon. Similar final counts do not imply equal experience: substantial delay can occur before the system clears.

At default demand, accumulation persists without a surge. Adding the surge raises final backlog from 2,816.5 to 4,028.5 people, while mean boarding fraction falls from 64.7% to 56.2%. Around 5,170 people board in either default-demand scenario, consistent with a service-limited system. Redistributing choices cannot offset sustained total inflow above total service. These results do not measure a real-world benefit of recommendations: no alternative recommendation or adoption policy has been compared.

**Conditional distribution collection.** A separate default-plus-surge collection recorded exactly 1,000,000 paired queue snapshots over 2,084 independently seeded trajectories, including a partial final trajectory. Sampling is every five seconds with five-minute bins. There are 7,400 entry/window/local-count cells; 5,666 have at least 30 matches. Cell counts across both entry views sum to 2,000,000 because each joint observation contributes once per view, not because it represents two independent observations. Histogram counts exactly match each cell's count, and accumulated costs are finite.

The lookup estimates

$$\widehat{P}(N_{\text{other}} = k \mid E = e, W = w, N_{\text{local}} = m) = \frac{h_{e,w,m}(k)}{n_{e,w,m}},$$

where $h$ is the matching histogram and $n$ its total. Conditional expected costs are stored cost sums divided by $n$. Histogram quantiles describe simulated variability, not confidence intervals. Thirty matches is a display safeguard, not a statistical guarantee. The table is generated under full-information choice behaviour, so it supports an occasional lookup user under that generating regime. Mass adoption changes behaviour and requires new policy simulations.

**Implementation verification and limits.** Model and API checks passed for NHPP integrals and pulse boundaries, stable Logit behaviour, position-loss costs, seeded reproducibility, conservation including walkers, capacities, conditional aggregation, transport parity and invalid requests. All 120 experimental trajectories satisfy arrivals equal the sum of boarded, queued and walking passengers at the horizon. These checks establish implementation consistency, not empirical validity or agreement with a separate SimPy implementation.

Results remain conditional on uncalibrated arrival, service and behavioural assumptions. High-demand states stretch the walking rule: spacing times combined queue length can imply distances much longer than the corridor. The actual engine serves passengers during their walk, while perceived cost uses a current-rate approximation and excludes existing walkers from observed queue lengths. Empty starts, a single surge and finite-horizon censoring further restrict generalisation. These results are stress experiments and should remain distinct from subsequently calibrated team results.